

# IBMFezNetworkMultiHopSyndrome

January 15, 2026

```
[1]: # =====  
# QUANTUM NETWORK MULTI-HOP EMULATION  
# =====  
# This notebook tests multi-hop quantum state transport across  
# multiple nodes on the IBM Fez topology.  
#  
# Uses the network_emulation package for modular functions.  
# =====  
  
import sys  
import numpy as np  
from pathlib import Path  
  
# Add parent directory to path for network_emulation package  
sys.path.insert(0, '..')  
  
# Import from network_emulation package  
from network_emulation import (  
    # Node utilities  
    Node, NODE_NAMES, NUM_NODES, INITIAL_STATES, STATE_LABELS,  
    CONNECTION_TUPLES, node_name, route_name,  
    # Config loading  
    load_nodes_config, load_routes_config,  
    # Circuit building (including new multi-hop function)  
    build_swapping_circuit, build_multihop_swapping_circuit,  
    # Experiment execution  
    initialize_backend, run_single_experiment, run_full_experiment,  
    # Visualization & export  
    plot_syndrome_matrix, plot_all_states_comparison, plot_state_averages_bar,  
    print_matrix_summary, print_full_comparison, save_results_to_json  
)  
  
print("Imports complete - using network_emulation package")  
print("  - build_multihop_swapping_circuit() now available")
```

```
Imports complete - using network_emulation package  
  - build_multihop_swapping_circuit() now available
```

```
[2]: # =====
# EXPERIMENT CONFIGURATION
# =====
# Set USE_REAL_HARDWARE = True to run on actual IBM quantum hardware
# Set USE_REAL_HARDWARE = False to use the FakeFcz simulator
# =====

USE_REAL_HARDWARE = True # Toggle between real hardware and simulator

# Experiment parameters
NUM_SHOTS = 4096          # Number of shots per circuit execution
OPTIMIZATION_LEVEL = 0    # Transpiler optimization (0 = no optimization)

# Configuration file paths
CONFIG_DIR = Path('../configs')
NODES_FILE = CONFIG_DIR / 'config_1_nodes.json'
ROUTES_FILE = CONFIG_DIR / 'config_1_routes.json'
OUTPUT_DIR = Path('../outputs/syndromes')

print("="*60)
print("EXPERIMENT CONFIGURATION")
print("="*60)
print(f"Hardware mode:      {'REAL IBM HARDWARE' if USE_REAL_HARDWARE else 'FakeFcz SIMULATOR'}")
print(f"Shots per circuit:    {NUM_SHOTS}")
print(f"Optimization level:    {OPTIMIZATION_LEVEL}")
print(f"Initial states:        {[STATE_LABELS[s] for s in INITIAL_STATES]}")
print(f"\nConfig files:")
print(f"  Nodes:  {NODES_FILE} (exists: {NODES_FILE.exists()})")
print(f"  Routes: {ROUTES_FILE} (exists: {ROUTES_FILE.exists()})")

=====
EXPERIMENT CONFIGURATION
=====

Hardware mode:      REAL IBM HARDWARE
Shots per circuit:  4096
Optimization level: 0
Initial states:      ['|0 ', '|1 ', '|+', '|- ']

Config files:
  Nodes:  ../configs/config_1_nodes.json (exists: True)
  Routes: ../configs/config_1_routes.json (exists: True)
```

```
[3]: # =====
# LOAD CONFIGURATIONS
# =====
```

```

# Load node and route configurations from JSON files
NODES = load_nodes_config(NODES_FILE)
TRANSPORT_ROUTES = load_routes_config(ROUTES_FILE)

print("="*60)
print("LOADED CONFIGURATIONS")
print("="*60)
print(f"\nNodes ({len(NODES)} total):")
for node_id, qubits in NODES.items():
    print(f"  Node {node_name(node_id)}: data={qubits['data']}, "
          f"  encoding={qubits['encoding']}, ancilla={qubits['ancilla']}")

print(f"\nRoutes ({len(TRANSPORT_ROUTES)} total)")
print(f"Connection pairs: {len(CONNECTION_TUPLES)}")

```

```

=====
LOADED CONFIGURATIONS
=====

```

Nodes (6 total):

```

Node A: data=[76], encoding=[61, 81], ancilla=[62, 82]
Node B: data=[77], encoding=[65, 85], ancilla=[66, 86]
Node C: data=[78], encoding=[69, 89], ancilla=[70, 90]
Node D: data=[96], encoding=[83, 103], ancilla=[84, 104]
Node E: data=[97], encoding=[87, 107], ancilla=[88, 108]
Node F: data=[98], encoding=[91, 111], ancilla=[92, 112]

```

Routes (30 total)

Connection pairs: 30

```

[4]: # =====
# INITIALIZE BACKEND
# =====

# Initialize quantum backend (FakeFez simulator or real IBM Fez)
backend, simulator, coupling_map, IS_REAL_HARDWARE = initialize_backend(
    USE_REAL_HARDWARE,
    secrets_path='../secrets/keys.json',
    simulation_method="matrix_product_state"
)

```

```

qiskit_runtime_service._discover_account:WARNING:2026-01-15 15:08:33,017:
Loading account with the given token. A saved account will not be used.

```

Connected to REAL backend: ibm\_fez

```

Qubits: 156
Status: active
Status: active

```

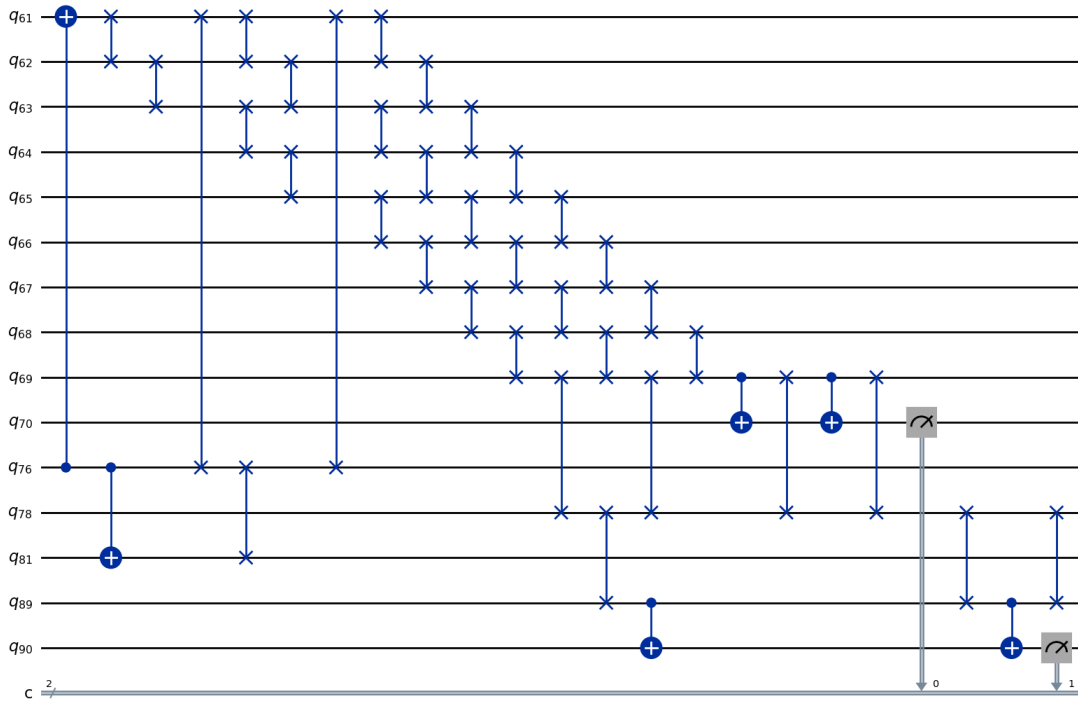
# 1 Run Experiments

The cells below execute the transport experiments. - **Single Test:** Quick test of one route - **Full Experiment:** All routes  $\times$  all initial states

```
[5]: # =====  
# QUICK TEST: Single Route  
# =====  
  
# Test a single route to verify circuit building  
test_circuit = build_swapping_circuit(  
    source=Node.A,  
    sink=Node.C,  
    initial_state='0',  
    nodes=NODES,  
    routes=TRANSPORT_ROUTES,  
    num_qubits=backend.num_qubits  
)  
  
print(f"Test circuit: Node A  $\rightarrow$  Node C, initial state |0 ")  
print(f"  Qubits: {test_circuit.num_qubits}")  
print(f"  Classical bits: {test_circuit.num_clbits}")  
print(f"  Gate count: {test_circuit.size()}")  
print(f"  Depth: {test_circuit.depth()}")  
  
# Draw the circuit  
test_circuit.draw('mpl', fold=-1, idle_wires=False)
```

```
Test circuit: Node A  $\rightarrow$  Node C, initial state |0  
  Qubits: 156  
  Classical bits: 2  
  Gate count: 42  
  Depth: 20
```

```
[5]:
```



## 2 Multi-Hop Circuit Testing

Test the new `build_multihop_swapping_circuit()` function which: - Takes a path of nodes (e.g.,  $A \rightarrow B \rightarrow D \rightarrow F$ ) - Optionally measures syndromes at intermediate nodes - Resets ancillas after mid-circuit measurements

```
[6]: # =====
# TEST 1: Multi-hop circuit with measurement at final node only (default)
# =====
# Path: A → B → D → F (3 hops)

multihop_circuit_1 = build_multihop_swapping_circuit(
    node_path=[Node.A, Node.B, Node.D, Node.F],
    initial_state='+',
    nodes=NODES,
    routes=TRANSPORT_ROUTES,
    num_qubits=backend.num_qubits,
    measure_at=None # Default: measure only at final node
)

print("Multi-hop circuit: A → B → D → F")
print(f"  Initial state: |+")
print(f"  Measurement: Final node only (F)")
```

```

print(f"  Qubits: {multihop_circuit_1.num_qubits}")
print(f"  Classical bits: {multihop_circuit_1.num_clbits}")
print(f"  Gate count: {multihop_circuit_1.size()}")
print(f"  Depth: {multihop_circuit_1.depth()}")

```

Multi-hop circuit:  $A \rightarrow B \rightarrow D \rightarrow F$   
 Initial state:  $|+\rangle$   
 Measurement: Final node only (F)  
 Qubits: 156  
 Classical bits: 2  
 Gate count: 73  
 Depth: 36

```

[7]: # =====
# TEST 2: Multi-hop with intermediate measurements
# =====
# Path:  $A \rightarrow B \rightarrow D \rightarrow F$ , measure at B and F

multihop_circuit_2 = build_multihop_swapping_circuit(
    node_path=[Node.A, Node.B, Node.D, Node.F],
    initial_state='+',
    nodes=NODES,
    routes=TRANSPORT_ROUTES,
    num_qubits=backend.num_qubits,
    measure_at=[Node.B, Node.F] # Measure at intermediate B and final F
)

print("Multi-hop circuit:  $A \rightarrow B \rightarrow D \rightarrow F$ ")
print(f"  Initial state:  $|+\rangle$ ")
print(f"  Measurement points: B (intermediate), F (final)")
print(f"  Classical bits: {multihop_circuit_2.num_clbits} (2 per measurement_
    ↪point)")
print(f"    c[0:2] = syndromes at Node B")
print(f"    c[2:4] = syndromes at Node F")
print(f"  Gate count: {multihop_circuit_2.size()}")
print(f"  Depth: {multihop_circuit_2.depth()}")

```

Multi-hop circuit:  $A \rightarrow B \rightarrow D \rightarrow F$   
 Initial state:  $|+\rangle$   
 Measurement points: B (intermediate), F (final)  
 Classical bits: 4 (2 per measurement point)  
   c[0:2] = syndromes at Node B  
   c[2:4] = syndromes at Node F  
 Gate count: 85  
 Depth: 44

```

[8]: # =====
# TEST 3: Measure at ALL intermediate nodes

```

```

# =====
# Path: A → B → D → F, measure at every stop (B, D, F)

multihop_circuit_3 = build_multihop_swapping_circuit(
    node_path=[Node.A, Node.B, Node.D, Node.F],
    initial_state='0',
    nodes=NODES,
    routes=TRANSPORT_ROUTES,
    num_qubits=backend.num_qubits,
    measure_at=[Node.B, Node.D, Node.F]  # Measure at all nodes along the path
)

print("Multi-hop circuit: A → B → D → F")
print(f"  Initial state: |0 ")
print(f"  Measurement points: B, D, F (all stops)")
print(f"  Classical bits: {multihop_circuit_3.num_clbits}")
print(f"    c[0:2] = syndromes at Node B (with ancilla reset)")
print(f"    c[2:4] = syndromes at Node D (with ancilla reset)")
print(f"    c[4:6] = syndromes at Node F (final, no reset)")
print(f"  Gate count: {multihop_circuit_3.size()}")
print(f"  Depth: {multihop_circuit_3.depth()}")

```

```

Multi-hop circuit: A → B → D → F
  Initial state: |0
  Measurement points: B, D, F (all stops)
  Classical bits: 6
    c[0:2] = syndromes at Node B (with ancilla reset)
    c[2:4] = syndromes at Node D (with ancilla reset)
    c[4:6] = syndromes at Node F (final, no reset)
  Gate count: 96
  Depth: 51

```

```

[9]: # # =====
# # RUN: Execute multi-hop circuit with intermediate measurements
# # =====

from qiskit import transpile

# # Transpile the circuit with intermediate measurements
# transpiled_multihop = transpile(
#     multihop_circuit_2,
#     backend,
#     initial_layout=list(range(backend.num_qubits)),
#     optimization_level=1
# )

# # Run on simulator

```

```

# job = simulator.run(transpiled_multihop, shots=NUM_SHOTS)
# multihop_counts = job.result().get_counts()

# print("Multi-hop A → B → D → F results (measured at B and F):")
# print(f" Total shots: {NUM_SHOTS}")
# print(f"\nSyndrome counts (format: 'FF BB' where FF=Node F, BB=Node B):")
# for outcome, count in sorted(multihop_counts.items(), key=lambda x: -x[1])[:
↪8]:
#     pct = count / NUM_SHOTS * 100
#     print(f" '{outcome}': {count:4d} ({pct:5.1f}%)")

# # Calculate '0000' rate (no errors at any measurement point)
# no_error_count = multihop_counts.get('0000', multihop_counts.get('00 00', 0))
# print(f"\nAll syndromes '00' rate: {no_error_count/NUM_SHOTS*100:.1f}%")

```

### 3 Exploring Syndromes through multi-path settings

```

[10]: node_path_1 = [Node.A, Node.D, Node.C, Node.F]
node_path_2 = [Node.A, Node.E, Node.C, Node.F]

multihop_circuit_path_1 = build_multihop_swapping_circuit (
    node_path=node_path_1,
    initial_state='0',
    nodes=NODES,
    routes=TRANSPORT_ROUTES,
    num_qubits=backend.num_qubits,
    measure_at=[Node.D, Node.F]
)

multihop_circuit_path_2 = build_multihop_swapping_circuit (
    node_path=node_path_2,
    initial_state='0',
    nodes=NODES,
    routes=TRANSPORT_ROUTES,
    num_qubits=backend.num_qubits,
    measure_at=[Node.E, Node.F]
)

```

```

[11]: transpiled_multihop_path_1 = transpile(
    multihop_circuit_path_1,
    backend,
    initial_layout=list(range(backend.num_qubits)),
    optimization_level=1
)

transpiled_multihop_path_2 = transpile(

```

```

    multihop_circuit_path_2,
    backend,
    initial_layout=list(range(backend.num_qubits)),
    optimization_level=1
)

```

```

[12]: # Run path 1 experiment
if IS_REAL_HARDWARE:
    from qiskit_ibm_runtime import SamplerV2
    sampler = SamplerV2(backend)
    job = sampler.run([transpiled_multihop_path_1], shots=NUM_SHOTS)
    print(f"Job submitted to {backend.name}: {job.job_id()}")
    result = job.result()
    # Use join_data() to combine all ClassicalRegisters into one bitstring
    multihop_counts = result[0].join_data().get_counts()
else:
    job = simulator.run(transpiled_multihop_path_1, shots=NUM_SHOTS)
    multihop_counts = job.result().get_counts()

```

Job submitted to ibm\_fez: d5kkihbtlojc73f6u5l0

```

[13]: print(f"\nMulti-hop {node_path_1} results (measured at D and F):")
print(f"    Total shots: {NUM_SHOTS}")
print(f"\nSyndrome counts (format: 'DD FF' where DD=Node D, FF=Node F):")
for outcome, count in sorted(multihop_counts.items(), key=lambda x: x[0]):
    pct = count / NUM_SHOTS * 100
    print(f"    '{outcome}': {count:4d} ({pct:5.1f}%)")

# Calculate '0000' rate (no errors at any measurement point)
no_error_count = multihop_counts.get('0000', multihop_counts.get('00 00', 0))
print(f"\nAll syndromes '00' rate: {no_error_count/NUM_SHOTS*100:.1f}%")

```

Multi-hop [<Node.A: 0>, <Node.D: 3>, <Node.C: 2>, <Node.F: 5>] results (measured at D and F):

Total shots: 4096

Syndrome counts (format: 'DD FF' where DD=Node D, FF=Node F):

```

'0000': 2353 ( 57.4%)
'0001':  134 (  3.3%)
'0010':   94 (  2.3%)
'0011':   25 (  0.6%)
'0100':  285 (  7.0%)
'0101':  244 (  6.0%)
'0110':   42 (  1.0%)
'0111':   27 (  0.7%)
'1000':  340 (  8.3%)
'1001':   39 (  1.0%)

```

```

'1010': 90 ( 2.2%)
'1011': 23 ( 0.6%)
'1100': 250 ( 6.1%)
'1101': 50 ( 1.2%)
'1110': 29 ( 0.7%)
'1111': 71 ( 1.7%)

```

All syndromes '00' rate: 57.4%

```

[14]: # Run path 2 experiment
if IS_REAL_HARDWARE:
    from qiskit_ibm_runtime import SamplerV2
    sampler = SamplerV2(backend)
    job = sampler.run([transpiled_multihop_path_2], shots=NUM_SHOTS)
    print(f"Job submitted to {backend.name}: {job.job_id()}")
    result = job.result()
    # Use join_data() to combine all ClassicalRegisters into one bitstring
    multihop_counts = result[0].join_data().get_counts()
else:
    job = simulator.run(transpiled_multihop_path_2, shots=NUM_SHOTS)
    multihop_counts = job.result().get_counts()

```

Job submitted to ibm\_fez: d5kkil2vcahs73a23u9g

```

[15]: print(f"\nMulti-hop {node_path_2} results (measured at E and F):")
print(f"  Total shots: {NUM_SHOTS}")
print(f"\nSyndrome counts (format: 'EE FF' where EE=Node E, FF=Node F):")
for outcome, count in sorted(multihop_counts.items(), key=lambda x: x[0]):
    pct = count / NUM_SHOTS * 100
    print(f"  '{outcome}': {count:4d} ({pct:5.1f}%)")

# Calculate '0000' rate (no errors at any measurement point)
no_error_count = multihop_counts.get('0000', multihop_counts.get('00 00', 0))
print(f"\nAll syndromes '00' rate: {no_error_count/NUM_SHOTS*100:.1f}%")

```

Multi-hop [<Node.A: 0>, <Node.E: 4>, <Node.C: 2>, <Node.F: 5>] results (measured at E and F):

Total shots: 4096

Syndrome counts (format: 'EE FF' where EE=Node E, FF=Node F):

```

'0000': 2325 ( 56.8%)
'0001': 109 ( 2.7%)
'0010': 83 ( 2.0%)
'0011': 37 ( 0.9%)
'0100': 282 ( 6.9%)
'0101': 289 ( 7.1%)
'0110': 39 ( 1.0%)
'0111': 32 ( 0.8%)

```

```
'1000': 319 ( 7.8%)  
'1001': 41 ( 1.0%)  
'1010': 136 ( 3.3%)  
'1011': 28 ( 0.7%)  
'1100': 158 ( 3.9%)  
'1101': 61 ( 1.5%)  
'1110': 36 ( 0.9%)  
'1111': 121 ( 3.0%)
```

All syndromes '00' rate: 56.8%

[ ]: